

Jour 5 — Intégration & Myhill–Nerode

Modules : `pipeline.py`, `myhill_nerode.py` • Barème : 3 pts (+1)

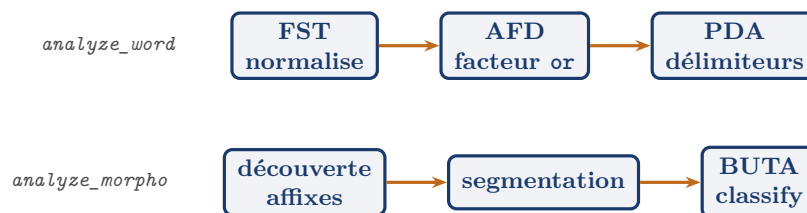
Contexte

Deux visages. **Assembler** : les étages, jusqu'ici isolés, **coopèrent** dans un **pipeline**. **Boucler la boucle théorique** : on revient sur la question du jour 1 — « combien d'états faut-il vraiment ? » — avec l'outil qui y répond exactement, la **congruence de Myhill–Nerode**.

Notions nouvelles du jour

1. Intégration : deux chaînes, un même cœur

Le pipeline appelle les apps, quiinstancient `formlang`. Deux chaînes de traitement :



On évalue l'**architecture** (chaque étage instancie le cœur), pas un nouvel algorithme.

2. La congruence de Myhill–Nerode

$u \approx_L v$ si, **pour tout** suffixe w , $uw \in L \Leftrightarrow vw \in L$: une fois u ou v lu, l'*avenir* (ce qu'on pourra encore accepter) est identique. Équivalence **invariante à droite** — d'où l'on fait les **états** d'un automate.

Le théorème — le fil rouge se referme

L est régulier ssi $\approx_L$ a un nombre **fini** de classes, et ce nombre (l'**indice**) est **exactement** le nombre d'états de l'AFD **minimal**. Minimiser (jour 1), c'était calculer ces classes.

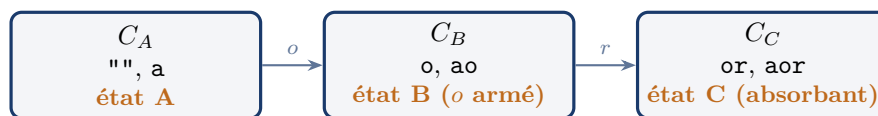
3. Le calcul en pratique : signatures sur suffixes témoins

On approxime « pour tout suffixe » par un **jeu fini** de suffixes distinctifs $S = \{\varepsilon, r, or\}$. La **signature** d'un mot est $(1[ws \in L_1])_{s \in S}$. Deux mots de même signature sont dans la même classe :

Trois signatures = trois classes

mot w	$w\varepsilon$	wr	wor	classe
""	0	0	1	A
a	0	0	1	A
o	0	1	1	B
ao	0	1	1	B
or	1	1	1	C
aor	1	1	1	C

(1 = « contient or ».) Signatures $(0, 0, 1), (0, 1, 1), (1, 1, 1)$: exactement **3 classes** $\Rightarrow$ 3 états de l'AFD minimal. `nerode_classes` fait ce regroupement (test `test_nerode_trois_classes`).



4. Ouverture : Myhill–Nerode pour les arbres (bonus)

Deux sous-arbres sont équivalents s'ils donnent le même verdict **dans tout contexte** (arbre « à trou ») : fondement de la minimisation d'un BUTA, et lien direct avec le hash-consing du jour 3 (partager, c'est identifier des sous-structures équivalentes).

A. Théorie (à rédiger) — 1 pt

Q5.1 (0,5)

Énoncer Myhill–Nerode ; relier le nombre de classes de L_1 à l'AFD minimal.

Q5.2 (0,5)

Critère « indistinguables par tous les suffixes témoins » et lien avec la fusion d'états.

B. Pratique — 2 pts

E5.1 (1)

`analyze_word` (FST $\rightarrow$ AFD $\rightarrow$ PDA) et `analyze_morpho` (découverte $\rightarrow$ segmentation $\rightarrow$ BUTA).
Tests pipeline.

E5.3 (1)

`nerode_classes` : regrouper par signature de suffixes. *Test* : `test_nerode_trois_classes`.

Sortie de référence

```
$ pytest -q
28 passed
$ python pipeline.py -word 4or
  normalisé(FST) : aor  facteur_or(AFD) : True  délimiteurs_ok(PDA)
: True
```

Mini-barème

Item	Détail	Pts
Q5.1–Q5.2	Myhill–Nerode + lien minimisation	1
E5.1	pipeline (mot + morpho)	1
E5.3	classes de Nerode	1
Bonus	mesure hash-consing sur 10 000 mots	+1

Gate (règle non négociable)

Le pipeline **appelle les apps** (quiinstancient le cœur) ; il ne reprogramme aucun automate.

Références

Hopcroft, Motwani, Ullman — Myhill–Nerode, minimisation. • Comon *et al.*, *TATA* (2007) — congruence pour les arbres (bonus).